Local Sandbox

A visual setup guide for Oracle-guided iOS tools, bounded local reasoning, and verifiable execution.

Oracle guidance	Shortcuts executor	Scoped file access	Xcode companion

---

BUILD ORDER

1. Folder
2. Dispatcher
3. Provider
4. Runner
5. Receipt
6. Siri

BUDGETS

6 steps initially
3 failures maximum
2 identical failures stop
One model at a time

IDENTITY

Web: ChatGPT session
Native: Apple sign-in
Folder: separate permission

1

Use one executor and interchangeable reasoning

Oracle prepares a compact task. A micro-model or frontier provider proposes one tool call. A deterministic dispatcher validates and executes it. Readback evidence becomes a receipt. Siri is a voice entry and Apple-owned orchestrator.

2

Open the private observation dashboard

https://local-sandbox.candy-lyric-5390.chatgpt.site - use your ChatGPT session. The online sandbox saves text files, policy, progress, and receipts. It is a separate source workspace, not a live mount of iCloud Drive.

3

Choose the iPhone path

For Shortcuts-only use, configure the recipes on the following pages. For native use, open ios/LocalSandbox.xcodeproj in Xcode, choose your developer team, enable Sign in with Apple, build to your device, then select your LocalSandbox folder.

What is ready, and what is still required

The web application and native source are prepared. The native app has not been compiled or run on an Apple device in this Linux environment. Signed shortcuts, Apple provisioning, device acceptance, external local-provider actions, API credentials, and App Review remain release gates.

01 / Prepare Workspace

Create the approved folders and the first test file.

Sequential	Compact context	Typed results	User-owned policy

SHORTCUT

Prepare Workspace

ROLE

Create the root

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

1 **Folder**
In Files create iCloud Drive / Shortcuts / LocalSandbox. This is the only folder the dispatcher uses.

2 **Create Folder**
Create config, state, workspace, logs, and output inside it.

3 **Text**
Status: pending

4 **Save File**
Save as workspace/test.txt. Disable overwrite.

5 **Dictionary**
Return root = LocalSandbox and ready = true.

Acceptance check
All five folders exist and workspace/test.txt contains Status: pending.

02 / Tool Dispatcher

Validate one tool call, apply it, and return a compact result.

Sequential	Compact context	Typed results	User-owned policy

SHORTCUT

Tool Dispatcher

ROLE

Execute one action

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

1. **Get Dictionary from Input**
   Read tool, arguments, run_id, and call_id. Reject missing fields.

2. **If**
   Allow only file.read, file.list, file.create, file.append, file.patch, device.clipboard_get, device.clipboard_set, and device.notify.

3. **Match Text**
   Allow only workspace, config, state, logs, or output relative paths. Reject absolute paths, backslashes, empty segments, dot and dot-dot segments, and encoded path characters. Never accept a root from the model.

4. **Get File from Folder**
   Resolve below the fixed LocalSandbox folder. file.read uses Get Text; file.list uses Get Contents of Folder and returns names. Limit reads to 8,000 characters.

5. **Match Text / Replace Text**
   For patches, reread the current file and compare the expected hash where available. Require exactly one literal old-text match. Disable regular expressions in Replace Text.

6. **Save File**
   Create refuses existing files. Before append or patch, save a backup in output. Perform one sequential mutation, then reread to verify.

7. **Show Notification / Clipboard**
   Dispatch the requested device action. Clipboard reads occur only in a user-initiated run.

Tool Dispatcher / continued

Validate one tool call, apply it, and return a compact result.

Same shortcut	Continue in order	Validate	Record evidence

SHORTCUT

Tool Dispatcher

ROLE

Execute one action

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

8

Dictionary

Return ok, tool, call_id, result, error_code, retryable. Unknown tools return INVALID_ARGUMENT.

Acceptance check

Traversal and ambiguous patches are rejected. A valid patch changes exactly one match.

03 / Agent Step - Local

Ask Locally AI for one structured action.

Sequential	Compact context	Typed results	User-owned policy

SHORTCUT

Agent Step - Local

ROLE

Micro-model reasoning

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

1 **Get Dictionary from Input**

Read goal, policy, capsule, and last_result.

2 **Text**

Use config/oracle.md, relevant tool schemas, the goal, capsule, and latest result. Stay within contextChars. Character count is a proxy; measure tokens in a runtime with a tokenizer.

3 **Ask Locally AI**

Select an installed local model. Pass the previous Text output. This requires Locally AI to expose its Shortcuts action. For LM Link use its verified action or endpoint through the same adapter.

4 **Get Dictionary from Input**

Require a JSON object with kind = tool or final. Reject prose and markdown fences.

5 **Stop and Output**

Return the action. The model never executes it.

Acceptance check

One valid JSON action is returned. Invalid JSON consumes a failure and gets one repair attempt.

04 / Agent Step - OpenAI

Use a protected Responses API relay after cloud consent.

Sequential	Compact context	Typed results	User-owned policy

SHORTCUT

Agent Step - OpenAI

ROLE

Frontier guidance

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

1 **Get Dictionary from Input**

Receive goal, capsule, latest result, and optional function-call continuation.

2 **If**

Check cloudAllowed before sending task or file text. Otherwise return CLOUD_NOT_AUTHORIZED.

3 **URL**

Use your authenticated HTTPS relay. Native API secrets stay in Keychain; a shared Shortcut must never contain your OpenAI API key.

4 **Get Contents of URL**

POST JSON. The relay calls https://api.openai.com/v1/responses with strict function schemas and parallel_tool_calls = false. Select a model available to your API project.

5 **Get Dictionary Value**

Inspect output. A function_call has name, arguments, and call_id. Return function_call_output with the original call_id during continuation.

6 **Stop and Output**

Normalize to the local tool/final contract. No silent provider switch after an uncertain mutation.

Acceptance check

Cloud-disabled runs send nothing. A configured relay preserves the original tool call ID.

05 / Agent Step - Apple

Use the companion app’s Foundation Models action.

Sequential	Compact context	Typed results	User-owned policy

SHORTCUT

Agent Step - Apple

ROLE

Apple Intelligence

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

1

Run App Intent

Select Local Sandbox → Plan One Step after installing the native companion.

2

Text

Pass the compact goal, state, relevant tools, and last result. Reserve space for output.

3

If

Check model availability. When unavailable, return its reason and select an approved fallback.

4

Get Dictionary from Input

Validate the same action contract. The dispatcher owns execution.

Acceptance check

The provider reports availability. Siri starts actions; Foundation Models performs the model step.

06 / Local Sandbox

Run sequential actions, repair failures, and checkpoint progress.

Sequential	Compact context	Typed results	User-owned policy

SHORTCUT

Local Sandbox

ROLE

Own the agent loop

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

1

Shortcut Input / Ask for Input

Accept text from Siri, the Share Sheet, or the dashboard. Generate UUID. Load config/policy.json.

2

Get Battery Level

Record initial level. Before each step, stop below batteryReserve or when observed level drop reaches batteryBudget. Charging can mask consumption; this measures level drop only.

3

Save File

Write state/active.json with run_id, task, step, failure counts, and last completed call ID.

4

Repeat

Repeat up to maxSteps. Call one selected Agent Step. Never run concurrent model steps.

5

If / Run Shortcut

For kind = tool call Tool Dispatcher. Record its call ID and result before another model call.

6

If

Retry transient reads once. Repair invalid JSON once. Reread on patch mismatch. Permission denial blocks. Stop at 3 failures or 2 identical failures. Never blindly retry an uncertain mutation.

7

Text

Compact to goal, completed steps, next action, file versions, constraints, and failures. Target a 200–500 token capsule; keep the trace on disk.

Local Sandbox / continued

Run sequential actions, repair failures, and checkpoint progress.

Same shortcut	Continue in order	Validate	Record evidence

SHORTCUT

Local Sandbox

ROLE

Own the agent loop

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

8

If

For kind = final reread changed files to verify. Without evidence return blocked.

9

Run Shortcut

Run Finish Run for every exit, including step or failure exhaustion.

Acceptance check

Read → patch → reread → verified receipt. Two identical failures terminate the run.

07 / Finish Run

Save the evidence and say what happened.

Sequential	Compact context	Typed results	User-owned policy

SHORTCUT

Finish Run

ROLE

Receipt and notification

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

1

Dictionary

Create id, goal, status, origin = shortcut, summary, steps, failures, durationMs, createdAt, trace. Status is verified, blocked, or cancelled.

2

Save File

Write logs/<run-id>.json. Raw traces stay outside the next prompt.

3

Show Notification

If enabled, show status and a short summary.

4

Stop and Output

Return the receipt. Use Import receipt in the dashboard. Native runs retain receipts in the selected folder.

Acceptance check
A blocked run also produces a receipt with its reason.

08 / Siri + Share Sheet

Expose the same runner through voice and sharing.

Sequential	Compact context	Typed results	User-owned policy

SHORTCUT

Siri + Share Sheet

ROLE

Launch from anywhere

INPUTS

Goal, compact state, latest result, saved policy.

OUTPUT

One action or one typed result. Keep detailed traces on disk.

APPROVAL

Install/configure on iPhone. Grant folder and notification access through iOS. Never infer permission from model text.

1 **Shortcut Details**

Enable Show in Share Sheet for text. Name the runner Local Sandbox.

2 **Siri**

Say “Siri, Local Sandbox.” Siri launches the shortcut with its existing permissions.

3 **Settings**

On a supported iPhone assign Shortcuts → Local Sandbox to the Action Button.

4 **Add to Home Screen**

In Safari use Share → Add to Home Screen for this dashboard. Use the native app for Apple sign-in and scoped iCloud access.

Acceptance check

Voice and shared text use the same runner, policy, and limits.

Native setup / Xcode

Apple identity, scoped iCloud access, and local execution are separate capabilities.

iOS 26+ target	SwiftUI	App Intents	Foundation Models

OPEN

ios/LocalSandbox.xcodeproj

SIGNING

Select an Apple Developer team and a unique bundle ID.

CAPABILITIES

Sign in with Apple is declared. The system folder picker grants scoped access.

RELEASE GATE

Run on a supported iPhone. Simulator battery readings are unavailable, so resource admission blocks there.

1

Build and sign

Use Xcode with the iOS 26 SDK or later. Set DEVELOPMENT_TEAM and a unique PRODUCT_BUNDLE_IDENTIFIER in Signing & Capabilities. The project contains no signing identity or private key.

2

Sign in with Apple

The app uses AuthenticationServices, checks the authorization state, and saves the opaque user ID in Keychain. This is a local identity feature. It does not authenticate the separate Sites dashboard or create a remote Apple-backed account.

3

Select your sandbox

In native Settings choose iCloud Drive / Shortcuts / LocalSandbox through the system folder picker. The app stores a scoped bookmark and coordinates file reads and writes. A revoked or unavailable folder blocks execution.

4

Configure a provider

Apple Intelligence requires a supported device and enabled model availability. OpenAI uses your own project key and model ID from Keychain after cloud consent. Locally AI or LM Link runs through an installed named Shortcuts adapter.

5

Test the actual device

Run the model contract test against workspace/test.txt. Verify file contents, app receipt, interruption behavior, permission denial, battery reserve, cancellation, and thermal admission. Inspect the checkpoint after a crash before starting another run.

Resource policy / exact meaning

Your sliders authorize stop thresholds and model context choices, not OS-level resource reservations.

Consent	Measurement	Admission	Cancellation

DEFAULT POLICY

15% battery drop
20% reserve
35% app memory target
6 steps
3 failures

CLOUD

Off by default. API configuration alone does not authorize a cloud call.

CONTEXT

6,000 characters is a proxy, not a guaranteed token count. Measure tokenizer counts in qualified runtimes.

1

Battery

The runner reads the current battery percentage before and during a model step. It stops below the reserve or at the observed level-drop budget. While charging, that drop is not a measurement of consumed energy.

2

Memory

The native monitor reads this app process footprint relative to physical memory. It does not observe separate Foundation Models services or another local-provider app. iOS owns allocation and can terminate work earlier.

3

Thermal and cancellation

Serious or critical thermal pressure requests cancellation. Cooperative cancellation can take time; the runner rechecks limits before a tool runs. The web dashboard never displays fabricated live device telemetry.

4

Failure recovery

Invalid JSON and patch conflicts return to the model within the failure budget. Edits require a fresh hash and one literal old-text match. Call journals record pending before mutation and refuse uncertain replays. An interrupted run must be reviewed.

5

Evidence-driven improvement

Record model/runtime, device, exact task, policy, verified outcome, wall time, and errors. Change one prompt or schema parameter at a time. Promote a larger budget only after user authorization. Do not infer local model quality from a deterministic web check.

Release / every aspect is a goal

The complete goal map is also available in the dashboard as an exportable checklist.

Web	Device	Apple	Distribution

DISTRIBUTION

No App Store submission is claimed. Apple signing, store metadata, screenshots, privacy labels, support details, and App Review are required.

REPOSITORY

Source is downloadable. A publication script is included. GitHub publication is reported only after a successful remote push.

1. **Oracle guidance**
   Task decomposition, compact instructions, recovery advice, and evidence-based completion. Status: Implemented.
2. **Observation dashboard**
   Persistent policy, setup progress, online text workspace, and receipt inspection. Status: Implemented.
3. **Shortcuts stack**
   Eight visual recipes for executor, runner, providers, receipts, and Siri. Status: Device setup.
4. **Editable sandbox**
   Online file tools and native access to a user-selected iCloud folder. Status: Native validation.
5. **Resource consent**
   Battery-drop budget, reserve, memory target, steps, and cloud consent. Status: Implemented.
6. **Context engineering**
   One action per turn, bounded results, state capsule, single model. Status: Implemented.
7. **Retry and recovery**
   Typed failures, patch rereads, no blind mutation retries, terminal budgets. Status: Implemented.
8. **OpenAI provider**
   Server-side relay gated by cloud consent, runtime API key, and model ID. Status: Credentials needed.

Release goals / continued

Preserve the same execution contract across every provider.

Track	Test	Approve	Release

READINESS

Implemented web features and prepared native source are different evidence levels.

DO NOT SKIP

Real device checks and Apple account configuration.

9 **Locally AI / LM Link**

Named Shortcuts adapter using the external app’s installed integration. Status: Device setup.

10 **Apple Intelligence**

Availability-aware native Foundation Models provider. Status: Native validation.

11 **Siri**

App Intents and a named runner with system-owned orchestration. Status: Native validation.

12 **Apple sign-in**

Native AuthenticationServices and credential-state check. Requires a signed app and developer capability. Status: Apple setup.

13 **iCloud connection**

System folder picker with scoped access, separate from identity. Status: Apple setup.

14 **Xcode / App Store**

Native source and build configuration. Signing, device tests, and App Review are pending. Status: Release gate.

15 **GitHub publication**

Source export and publication script; repository creation depends on connected capability. Status: Release gate.

Sources / implementation notes

Primary documentation checked 10 September 2026. Source sections are named for direct verification.

Apple	OpenAI	Reference guide	Traceable

REFERENCE

User-supplied iOS_ChatGPT_Local_Agent_Setup_Guide.pdf, pp. 1-5. Its structure and bounded loop are retained; no model availability claims were assumed.

SDK BASELINE

The project targets documented iOS 26 APIs. Later system features must be qualified on the actual device.

1

Apple: run a shortcut from a URL

Shortcuts User Guide, “Run a shortcut from a URL.” https://support.apple.com/guide/shortcuts/run-a-shortcut-from-a-url-apd624386f42/ios

2

Apple: cross-app orchestration

WWDC26 Apple Intelligence Group Lab, 00:08:18 cross-app intent routing; 00:13:26 background limits; 01:00:09 on-device context. https://developer.apple.com/videos/play/wwdc2026/8011/

3

Apple: Sign in with Apple for the web

Account Help, “Associate your website to your app.” A web Apple sign-in needs a Services ID associated with a primary App ID. The private Sites version uses platform authentication instead. https://developer.apple.com/help/account/capabilities/configure-sign-in-with-apple-for-the-web/

4

Apple: Foundation Models

Developer Documentation, Foundation Models framework overview. Availability must be checked at runtime. https://developer.apple.com/documentation/foundationmodels

5

OpenAI: function calling

API guide, “Function calling.” A model proposes a call; application code executes it. Strict schemas do not grant authority. https://developers.openai.com/api/docs/guides/function-calling

6

Implementation evidence

See docs/RELEASE.md, docs/ARCHITECTURE.md, tests/contracts.test.ts, the generated migration, and native SandboxTests.swift. Only completed tests are recorded as passing. No device, thermal, model-quality, or Apple-login result is fabricated.